D20A **Roll No.19**

Experiment No. 10

Aim:

To configure a Hyperledger Fabric test network, deploy JavaScript chaincode, and perform asset initialization, querying, creation, and ownership transfer.

Theory:

Hyperledger Fabric is a permissioned blockchain framework where participating organizations manage peers and validate transactions based on endorsement policies. Channels establish private communication and ledger-sharing environments among authorized members. The blockchain ledger maintains transaction history, while the world state database stores the latest asset information. Chaincode defines the business logic used to access and modify ledger data through Fabric transaction contexts.

In this experiment, a JavaScript chaincode models assets as JSON records containing an asset ID, color, size, owner, and appraised value. The initialization function inserts predefined assets into the ledger. Query operations retrieve individual assets or multiple records, while asset creation adds new entries and transfer operations update ownership information. Hyperledger Fabric processes transactions through ordering and validation before committing valid changes to the ledger and world state. The provided chaincode is included in this report for reference as source code; it was not executed during document preparation.

Steps:

1. Update the Ubuntu package index and install the required tools, including jq, curl, git, Node.js, npm, and Docker.
2. Clone the hyperledger/fabric-samples repository and enter the fabric-samples directory.
3. Download and run install-fabric.sh to obtain the Fabric binaries, samples, and Docker images.
4. Verify the available Fabric images with docker images | grep fabric and check the CLI using peer version.
5. Set PATH and FABRIC_CFG_PATH to the Fabric binaries and configuration directories.
6. Enter test-network and start the network while creating the mychannel channel with ./network.sh up createChannel.
7. Deploy the JavaScript asset-transfer chaincode named basic using ./network.sh deployCC.
8. Configure the peer environment variables and initialize the ledger by invoking InitLedger.
9. Query all assets with GetAllAssets, then read a specific record such as asset5 with ReadAsset.
10. Invoke CreateAsset to add a new asset and TransferAsset to change its owner; query the record to inspect its updated state.

Experiment No. 10

Code:

```
'use strict';
const { Contract } = require('fabric-contract-api');
class AssetTransfer extends Contract {
    async InitLedger(ctx) {
        const assets = [
            { ID: 'asset1', Color: 'blue', Size: 5, Owner: 'Tomoko', AppraisedValue: 300 },
            { ID: 'asset2', Color: 'red', Size: 5, Owner: 'Brad', AppraisedValue: 400 },
            { ID: 'asset3', Color: 'green', Size: 10, Owner: 'Jin Soo', AppraisedValue: 500 },
            { ID: 'asset4', Color: 'yellow', Size: 10, Owner: 'Max', AppraisedValue: 600 },
            { ID: 'asset5', Color: 'black', Size: 15, Owner: 'Adriana', AppraisedValue: 700 },
            { ID: 'asset6', Color: 'white', Size: 15, Owner: 'Michel', AppraisedValue: 800 },
        ];
        for (const asset of assets) {
            await ctx.stub.putState(
                asset.ID,
                Buffer.from(JSON.stringify(asset))
            );
        }
    }
    async CreateAsset(ctx, id, color, size, owner, appraisedValue) {
        const exists = await this.AssetExists(ctx, id);
        if (exists) {
            throw new Error(`Asset ${id} already exists`);
        }
        const asset = {
            ID: id,
            Color: color,
            Size: Number(size),
            Owner: owner,
            AppraisedValue: Number(appraisedValue),
        };
        await ctx.stub.putState(
            id,
            Buffer.from(JSON.stringify(asset))
        );
        return JSON.stringify(asset);
    }
    async ReadAsset(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
        if (!assetJSON || assetJSON.length === 0) {
            throw new Error(`Asset ${id} does not exist`);
        }
```

Experiment No. 10

```
        return assetJSON.toString();
    }
    async GetAllAssets(ctx) {
        const iterator = await ctx.stub.getStateByRange('', '');
        const allResults = [];
        let result = await iterator.next();
        while (!result.done) {
            const value = result.value.value.toString('utf8');
            if (value.length > 0) {
                allResults.push(JSON.parse(value));
            }
            result = await iterator.next();
        }
        await iterator.close();
        return JSON.stringify(allResults);
    }
    async TransferAsset(ctx, id, newOwner) {
        const assetString = await this.ReadAsset(ctx, id);
        const asset = JSON.parse(assetString);
        const previousOwner = asset.Owner;
        asset.Owner = newOwner;
        await ctx.stub.putState(
            id,
            Buffer.from(JSON.stringify(asset))
        );
        return previousOwner;
    }
    async AssetExists(ctx, id) {
        const assetJSON = await ctx.stub.getState(id);
        return assetJSON && assetJSON.length > 0;
    }
}
module.exports = AssetTransfer;
```

Listing 1. experiment10_chaincode.js

Output:

Experiment No. 10

```
[sudo] password for dipeshnawani1605:
warning: jq-1.8.2-1 is up to date -- skipping
warning: curl-8.22.0-1 is up to date -- skipping
warning: git-2.55.0-1 is up to date -- skipping
warning: nodejs-26.8.2-1 is up to date -- skipping
warning: docker-1:29.0.0-1 is up to date -- skipping
warning: docker-compose-5.5.1-1 is up to date -- skipping
warning: docker-buildx-0.37.1-1 is up to date -- skipping
resolving dependencies...
looking for conflicting packages...

Packages (1) npm-12.0.2-1

Total Download Size:   1.74 MiB
Total Installed Size:  9.39 MiB

:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
 npm-12.0.2-1-any                                    1783.0 KiB  1223 KiB/s 00:01 [##########################################################] 100%
error: failed retrieving file 'npm-12.0.2-1-any.pkg.tar.zst' from in.arch.hiranjan.co : The requested URL returned error: 404
(1/1) checking keys in keyring                                                     [##########################################################] 100%
(1/1) checking package integrity                                                   [##########################################################] 100%
(1/1) Loading package files                                                        [##########################################################] 100%
(1/1) checking for file conflicts                                                  [##########################################################] 100%
(1/1) checking available disk space                                                [##########################################################] 100%
:: Processing package changes...
(1/1) installing npm                                                               [##########################################################] 100%
Optional dependencies for npm:
    git: for dependencies using Git URL's [installed]
:: Running post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
                                                                                                                              18:52:33
```

Fig. 1. Terminal output showing package installation and dependency checks

```
~                                                                                                                             19:01:57
  curl -sSL0 https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh

~                                                                                                                             19:02:18
  chmod +x install-fabric.sh

~                                                                                                                             19:02:25
  ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

===> Changing directory to fabric-samples
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-samples main branch is intended to work with vecent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.16 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.16/hyperledger-fabric-linux-amd64-2.5.16.tar.gz
===> Will unpack to: /home/dipeshnawan1605/fabric-samples
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
  0      0   0      0   0      0      0       0                              0
100 121.8M 100 121.8M   0      0 669.1k      0   03506   03506         498.8k
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
===> Will unpack to: /home/dipeshnawani1605/fabric-samples
  % Total    % Received % Xferd  Average Speed  Time    Time    Time   Current
                                 Dload  Upload  Total   Spent   Left   Speed
  0      0   0      0   0      0      0       0                              0
100 31.65M 100 31.65M   0      0 305.8k      0   01:45   01:45         846.2k
==> Done.

Pull Hyperledger Fabric docker images
```

Fig. 2. Fabric installation script downloading binaries and samples

Experiment No. 10

```
~
  docker images | grep hyperledger
ghcr.io/hyperledger/fabric-baseos:2.5.16    109b4a15b282        257MB          83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB           123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16     807493f095f6       1.01GB           257MB
ghcr.io/hyperledger/fabric-orderer:2.5.16   1d918fc7e1e0        182MB          50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16      58979b65744b        232MB          66.8MB
hyperledger/fabric-baseos:2.5               109b4a15b282        257MB          83.2MB
hyperledger/fabric-baseos:2.5.16            109b4a15b282        257MB          83.2MB
hyperledger/fabric-baseos:latest            109b4a15b282        257MB          83.2MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB           123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB           123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB           123MB
hyperledger/fabric-ccenv:2.5                807493f095f6       1.01GB           257MB
hyperledger/fabric-ccenv:2.5.16             807493f095f6       1.01GB           257MB
hyperledger/fabric-ccenv:latest             807493f095f6       1.01GB           257MB
hyperledger/fabric-orderer:2.5              1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-orderer:2.5.16           1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-orderer:latest           1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-peer:2.5                 58979b65744b        232MB          66.8MB
hyperledger/fabric-peer:2.5.16              58979b65744b        232MB          66.8MB
hyperledger/fabric-peer:latest              58979b65744b        232MB          66.8MB
```

Fig. 3. Docker listing showing installed Hyperledger Fabric images

Experiment No. 10

```
~ export PATH=$PATH:$HOME/fabric-samples/bin

~ export FABRIC_CFG_PATH=$HOME/fabric-samples/config

~ peer version
peer:
 Version: v2.5.16
 Commit SHA: f871cf9
 Go version: go1.26.4
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Fig. 4. Fabric peer version and configured environment variables

```
+ res=0
+ configtxlator proto_encode --input /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/Org2MSPConfig.json --type common.Config --output
/home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/original_config.pb
+ configtxlator proto_encode --input /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/Org2MSPModified_config.json --type common.Config
--output /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/modified_config.pb
+ configtxlator compute_update --channel_id mychannel --original /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/original_config.pb --
updated /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/modified_config.pb --output /home/dipeshnawani1605/fabric-samples/test-network/cha
nnel-artifacts/config_update.pb
+ configtxlator proto_decode --input /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/config_update.pb --type common.ConfigUpdate --out
put /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/config_update.json
+ jq .
+ cat /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/config_update.json
+ echo '{"payload":'{"header":{"channel_header":'{"channel_id":,"mychannel","type":2},"data":{"config_update":{ "channel_id":' "mychannel", ' "isolate
d_data": '{},' "read_set":'{'"groups": '{{ "Application": '{'', "jaykication":'{"groups": '{{ "groups": '{},' "groups": '"{},' "mod_policy": '""" ',' "policie
s": '{ '"Admins":; '{ '"mod_policy":: """,' "policy": 'null,' "version"; '"0"}, "Endorsement": '{' "mod_policy":: \"",' "policy":; "null,' '"0", "v
ersion": '"0", },' "Readers": '{ '"mod_policy": '""",' "policy":: 'null,' "version"; '"0", }, "Writers": {{' "mod_policy":: """, "policy":: 'null, "v
null,' "version\" ;' '"0", }, ', "Writers": '{}, '"mod_policy": """, "policy":: """, "version":: '"0", }, '"groups": '{}, "mod_policy": '""",' "policee
": ', ""ord_policy": '""", ' "groups": '{}, '\"values": '{}, ' "version": "0", ', "groups": '{},' "version": "0", 'writiens": '""",' \"groups": '{', "m
"version": '"0", 'yulle,' "version": '"0", } '"writicies": '{},' "version": '{},' "groups": '{}, "mod_policy": '""", "groupes": '{},' "Endorsement": '{ '
s, '"Admins": {},' "Admins": '{}, '"mod_policy": '\""',' "policy": 'null,' "version": '"0" }, "Endorsement": '{' "mod_policy": '\"",' "weiiins": '{{' "moi
icy": null,' "version": "0",',' "Readers": {' "mod_policy":; """, "policy": 'null, } '"version": '"0", } "Writers": '{', "Wriers": '{},' "mod_policy": '""
"",' "policy": 'null,' "version": '!0", }}, ' "values": {' '"AnchorPeers": {' "mod_policy":' ""Admins', ' "value": '{' '"anchor_peers":" { {{,' "host: "
", '"peer0.org2.example.com", ' "port": '9051]] }}}, ' "version": '"0", }, '"MSP": '{'' "mod_policy": '"1"', "value"': 'null, '"version": '!0"'.!} }'
'on' },}}}}, "version": '"1" } }, '"mod_policy": '""",' "policies": '{},' "values": '{}, '"versies": '"0" '}
/home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
+ configtxlator proto_encode --input /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Enve
lope --output /home/dipeshnawani1605/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-10-13 19:36:50.687 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-10-13 19:36:50.701 IST 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
~/fabric-samples/test-network  main                                                                    19:36:50
```

Fig. 5. Channel configuration update completed and peer joined mychannel

Experiment No. 10

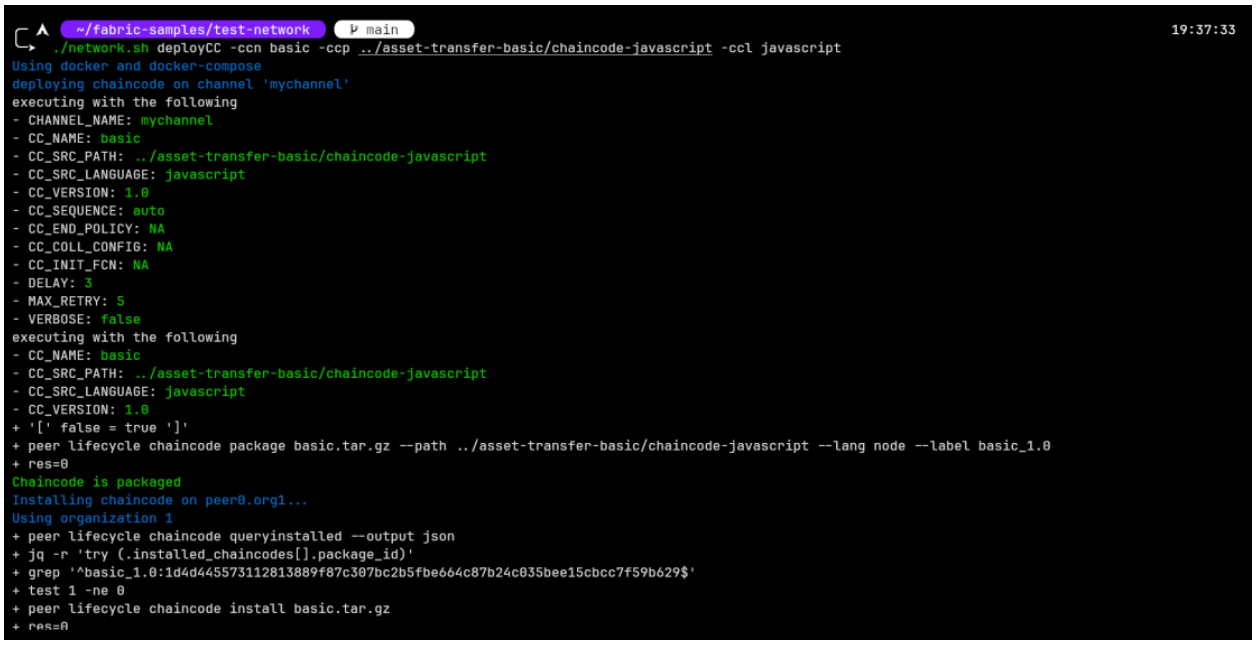


Fig. 6. JavaScript asset-transfer chaincode deployment progress



Fig. 7. InitLedger transaction completed successfully

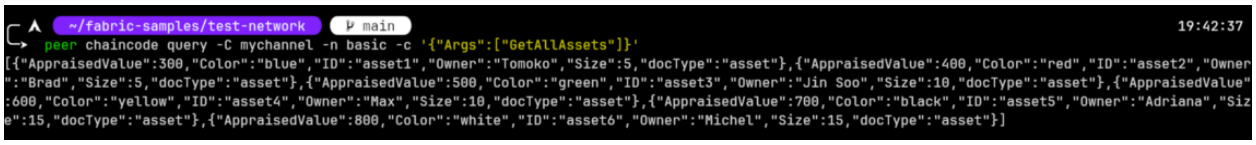


Fig. 8. Query output listing the initialized ledger assets

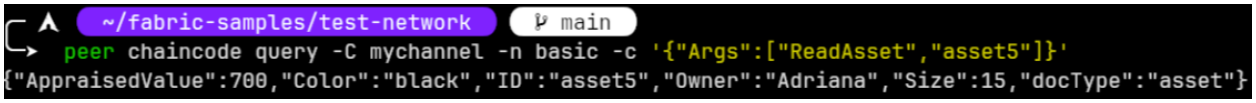


Fig. 9. ReadAsset query returning the details for asset5



Fig. 10. CreateAsset transaction creating asset7 for Dipesh Nawani

Experiment No. 10

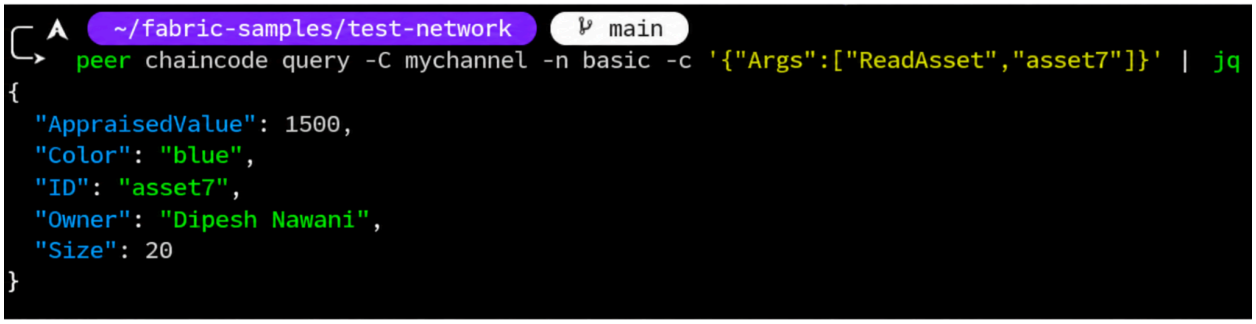


Fig. 11. ReadAsset query showing the newly created asset7



Fig. 12. TransferAsset transaction and updated owner of asset7

Conclusion:

The procedure and JavaScript chaincode for implementing an asset-management system using Hyperledger Fabric were documented. The provided screenshots illustrate terminal operations related to configuring the Fabric environment and interacting with the sample network. The chaincode presented in this report contains functions for initializing the ledger, creating assets, retrieving asset information, and transferring ownership. However, the chaincode was executed during the preparation of this document.